

```

// POST /api/score
// Body: the full score-params object (see README for the exact shape)
// Returns: the full computeScore() result object

const { computeScore, buildRecommendations, PURCHASE_CATEGORIES, getBand } = require('...')

module.exports = async (req, res) => {
  // CORS - allow Bubble (and anywhere else) to call this
  res.setHeader('Access-Control-Allow-Origin', '*');
  res.setHeader('Access-Control-Allow-Methods', 'POST, OPTIONS');
  res.setHeader('Access-Control-Allow-Headers', 'Content-Type');

  if (req.method === 'OPTIONS') {
    return res.status(200).end();
  }
  if (req.method !== 'POST') {
    return res.status(405).json({ error: 'Use POST' });
  }

  try {
    const body = req.body || {};

    // catDef is passed as a category id string from Bubble; resolve it to the real category
    const catId = body.categoryId || body.catDef;
    const catDef = PURCHASE_CATEGORIES.find(c => c.id === catId);
    if (!catDef) {
      return res.status(400).json({ error: `Unknown categoryId: ${catId}`, validId: catId });
    }

    const params = { ...body, catDef };
    const results = computeScore(params);
    const recommendations = buildRecommendations(params, results);
    const band = getBand(results.total);

    return res.status(200).json({
      total: results.total,
      band: { label: band.label, color: band.color },
      overridden: results.overridden,
      overrides: results.overrides,
      cashFlow: { points: results.cf.pts, max: 40, margin: results.cf.margin, pct: results.cf.pct },
      financialResilience: { points: results.cushionTotal, max: 35, savings: results.savings },
      debtLoadAdjustment: results.debtLoadAdjustment,
      baseScore: results.baseScore,
      financingQuality: { points: results.financingTotal, max: 25, breakdown: results.financingBreakdown }
    });
  } catch (err) {
    return res.status(500).json({ error: err.message });
  }
}

```

```
    recommendations,  
    raw: results, // full detail, in case Bubble needs anything else  
  });  
} catch (err) {  
  return res.status(500).json({ error: err.message, stack: err.stack });  
}  
};
```